

FAMM: Future-Aware Adaptive Memory Management Framework for Long-Term Autonomous LLM Agents

Suyash Vakhariya and Asmita Ipper
Department of Computer Engineering
PDEA College of Engineering
Savitribai Phule Pune University
Pune, Maharashtra, India
vakhariyasuyash@gmail.com
asmitaipper@gmail.com

Abstract—As Large Language Model (LLM) agents are increasingly deployed for long-term, unbounded operational tasks, managing their finite context windows becomes a critical architectural bottleneck. Existing Retrieval-Augmented Generation (RAG) and episodic memory systems typically rely on reactive heuristics—such as raw semantic similarity, uniform temporal decay, or static importance scoring—to determine which context to retain. These approaches fail in high-noise, temporally extended workflows due to context pollution and retrieval noise. We introduce the Future-Aware Adaptive Memory Management (FAMM) framework, an architecture that proactively estimates the future operational utility of each memory at write time. FAMM employs a lightweight, deterministic feature-based scoring function that bypasses LLM evaluation calls and drives a utility-conditioned decay mechanism coupled with a four-signal composite retrieval ranker incorporating semantic similarity, predicted utility, goal alignment, and temporal recency. We evaluate FAMM against four baseline architectures—Similarity-Only RAG, Importance Scoring, Uniform Ebbinghaus Decay, and FIFO eviction—on a synthetic research agent dataset scaled to 10,000 vector memories. Our results reveal a nuanced finding: while FAMM’s write-path architecture achieves effective storage consolidation and low-latency prediction without LLM calls, its goal-alignment retrieval signal introduces systematic over-filtering that degrades recall at all tested scales (NDCG@10 of 0.494 at $N=10,000$ vs. 0.750 for Importance Scoring). Ablation confirms that disabling goal-alignment restores competitive retrieval performance, providing a cautionary empirical result regarding multi-signal ranking in dense embedding spaces.

Index Terms—Large Language Models, Retrieval-Augmented Generation, Autonomous Agents, Vector Databases, Memory Management, Semantic Search, Complexity Analysis

I. INTRODUCTION

A. Motivation and the Rise of Autonomous Agents

The architectural maturation of Large Language Models (LLMs) has catalyzed a paradigm shift in artificial intelligence. Historically utilized as stateless text generators [1], contemporary LLMs increasingly serve as the cognitive reasoning engines for autonomous, goal-directed agents. These agents execute complex, multi-step workflows across software engineering, scientific research, and enterprise operations [2].

A defining characteristic of these deployments is temporal persistence. Unlike traditional zero-shot query systems, modern autonomous agents must maintain a coherent, evolving internal state across interactions spanning days or months. This requirement mirrors the operational realities of human cognitive systems, where contextual continuity is essential for trust, efficiency, and task resolution. However, the foundational architecture of transformer-based LLMs imposes a hard constraint on this autonomy: the finite context window.

B. The Bottleneck of Finite Context Windows

Transformer architectures rely on self-attention mechanisms with quadratic computational cost $O(N^2 \cdot d)$ with respect to sequence length N and model dimension d [3], [4]. Although engineering advances such as Ring Attention and sparse retrieval heads have expanded these boundaries to accommodate multi-million token windows, empirical evidence reveals persistent practical limitations. Injecting massive, unbounded operational histories into a static context window produces severe latency degradation.

Long-context models consistently exhibit the “lost in the middle” phenomenon: when critical instructions or operational constraints are buried within large blocks of noisy context, the attention mechanism fails to route them to the active reasoning stream. True long-term autonomy therefore cannot rely on brute-force context scaling; it requires structured, dynamic working memory architectures that selectively page critical information into the prompt space when the reasoning process demands it.

C. Limitations of Current Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [5] has emerged as the standard approach for managing external knowledge without continuous model fine-tuning. Dense embeddings project historical events into high-dimensional vector spaces, enabling agents to query external vector databases [6] for relevant context.

Despite its effectiveness in static, open-domain question-answering settings, traditional RAG rapidly deteriorates when applied to continuous, dynamic agent memory [7]. An autonomous agent’s memory stream is not a pristine encyclopedic corpus; it quickly fills with redundant observations, transient API failure responses, ephemeral thought traces, and obsolete operational plans.

Traditional RAG retrieves memories based purely on the cosine similarity between the query embedding and stored vectors [8]. In complex agentic workflows, however, high semantic similarity does not imply operational utility. An agent attempting to resolve a novel compiler error might retrieve thousands of semantically similar but practically useless logs of previous successful compilations. This naive semantic fetching produces context pollution and cascading hallucination loops [9].

D. The Need for Proactive Adaptive Memory

Recent cognitive architectures have introduced structural memory management to mitigate RAG failures. Frameworks such as Generative Agents [10] and MemGPT [11] attempt to filter noise by utilizing uniform temporal forgetting curves or assigning static importance metadata at creation time.

While these architectures represent meaningful advances, they remain fundamentally reactive and often computationally expensive. Relying on the LLM itself to assign importance scores to every incoming observation requires secondary inference calls that dominate the agent’s computational budget [12]. Moreover, static importance is highly contextual: a database password logged on day one is critically important until it is revoked on day two, at which point its importance should plummet—yet standard systems permanently shield it from eviction based on its initial high score.

Systems utilizing uniform forgetting curves introduce a complementary failure: they arbitrarily delete critical configuration data simply because it is chronologically old, treating a vital architectural constraint with the same temporal hostility as a transient web-scraping error. The field currently lacks an automated mechanism to decouple the chronological age of a memory from its operational utility.

E. Research Contributions

We hypothesize that memory management must transition from a reactive retrieval constraint to a proactive, write-time prediction. If an agent can estimate the *future operational utility* of a memory at the moment of creation—using lightweight deterministic scoring rather than expensive LLM evaluations—it can autonomously structure its own forgetting curve.

To test this hypothesis, we introduce the **Future-Aware Adaptive Memory Management (FAMM)** framework. FAMM intercepts the memory write stream using a deterministic weighted feature scorer to estimate future utility from five predictive signals: goal similarity, recency, access frequency, source type prior, and entity overlap. This score drives a utility-conditioned decay mechanism that shields high-value

memories from chronological eviction, and acts as a signal within a four-component composite retrieval ranker.

Our contributions are:

- 1) We formulate the FAMM architecture, detailing an in-process observer-pattern event bus that decouples memory management from the LLM’s primary reasoning loop, integrating write-time utility prediction and Union-Find background consolidation.
- 2) We introduce a utility-conditioned discrete decay rule where the effective decay rate is inversely proportional to predicted utility, providing up to $81\times$ differential temporal shielding between high- and low-utility memories.
- 3) We construct a four-signal composite retrieval ranker that synthesizes semantic similarity, predicted utility, goal alignment, and temporal recency into a unified ranking score.
- 4) We provide empirical evaluation on a synthetic dataset scaled to $N=10,000$ dense vectors. Through ablation studies and complexity analysis, we document a cautionary finding: the goal-alignment signal introduces systematic over-filtering that degrades recall at all tested scales, while the write-path architecture (utility prediction, decay, consolidation) operates as designed.

II. BACKGROUND

This section provides foundational concepts necessary to contextualize FAMM within the broader landscape of LLM memory management.

A. Transformer Memory and Context Windows

The transformer architecture [3] processes input as a fixed-length token sequence. The self-attention mechanism computes pairwise relationships across all positions with cost $O(N^2 \cdot d)$ where N is sequence length and d is model dimension. This fixed sequence constitutes the model’s *working memory*: only information physically present within the current token buffer is accessible to the reasoning process.

B. Working Memory vs. Long-Term Memory

Drawing from cognitive science [13], agent memory architectures distinguish between *working memory* (the active context window) and *long-term memory* (persistent external storage). Working memory is fast but severely capacity-constrained. Long-term memory offers unbounded capacity but introduces retrieval latency and noise. The fundamental challenge is bridging these tiers efficiently—paging critical data from long-term storage into working memory exactly when the reasoning process requires it.

C. Dense Embeddings and Vector Databases

Dense embedding models project variable-length text strings into fixed-dimensional vector spaces. Models such as all-MiniLM-L6-v2 [14] produce 384-dimensional L2-normalized vectors where geometric proximity corresponds to semantic similarity. Vector databases (e.g., ChromaDB, FAISS) maintain these embeddings in specialized indices—typically Hierarchical Navigable Small World (HNSW) graphs

[15]—that support approximate nearest neighbor (ANN) search in $O(\log N)$ time. The combination of dense embeddings with ANN indices forms the backbone of modern RAG systems.

III. RELATED WORK

A. LLM-Dependent Memory Architectures

Generative Agents [10] introduced a three-signal retrieval function combining recency, importance, and relevance. However, the importance score requires a full LLM inference call for every stored observation, creating a write-path bottleneck that scales linearly with memory volume.

MemGPT [11] models LLM memory management as an operating system, implementing virtual paging between a fixed main context and an unbounded archival store. Every page-in/page-out operation requires an LLM function call, introducing latency proportional to the frequency of context switches.

Reflexion [16] maintains episodic failure logs that the agent consults to avoid repeating mistakes. This approach excels at short-horizon error correction but provides no mechanism for long-term memory management or temporal decay.

Self-RAG [17] augments generation with self-reflective critique tokens that assess retrieval quality. While this improves generation fidelity, it adds inference overhead at every generation step and does not address the upstream problem of memory organization.

B. Memory Decay and Consolidation Systems

MemoryBank [18] applies the Ebbinghaus forgetting curve to LLM memory, implementing a uniform exponential decay across all stored records. Because this decay is content-agnostic, it arbitrarily evicts critical configuration data that happens to be chronologically old.

A-MEM [19] treats memories as self-organizing “Zettelkasten” notes where the agent creates and links memory entries. While this approach excels at organic knowledge structuring, every memory operation requires a full LLM call, limiting throughput to the rate of the underlying language model.

C. Advanced Retrieval Architectures

GraphRAG [20] builds knowledge graphs from document collections, enabling entity-relation traversal queries. The graph construction phase, however, requires extensive LLM extraction calls and does not support the real-time write throughput needed for continuous agent memory streams.

HippoRAG [21] mimics hippocampal indexing by maintaining a knowledge graph alongside dense vectors. The dual-index approach provides superior recall on knowledge-intensive benchmarks but introduces significant indexing overhead incompatible with high-velocity agentic data.

RAPTOR [22] constructs hierarchical cluster trees over document collections, enabling multi-level summarization retrieval. Each tree construction step requires LLM-based summarization, making the approach computationally prohibitive for streaming memory contexts.

D. Lightweight Memory Systems

LongMem [23] augments language models with a side-network memory module that caches recent activations. **LightMem** [24] injects retrieved context via cross-attention mechanisms. Both approaches embed memory into the model’s internal representations, sacrificing the auditability and transparency required for deployable autonomous systems.

E. How FAMM Differs

FAMM diverges from these paradigms by shifting memory management from a reactive retrieval constraint to a proactive, write-time prediction. Unlike MemGPT or Generative Agents, FAMM entirely removes the cognitive load of memory maintenance from the LLM, relying instead on a deterministic weighted scorer to predict utility. Unlike MemoryBank, FAMM’s temporal decay is not uniform; it is conditioned on predicted utility. Unlike GraphRAG, RAPTOR, and HippoRAG, FAMM operates on flat vector spaces with lightweight Union-Find clustering. Unlike LightMem and LongMem, FAMM’s memories remain fully discrete, transparent, and auditable. Table I provides a structured comparison.

IV. PROBLEM STATEMENT

Definition 1 (Agent Memory Record). *A memory record is a tuple $M = (s, t_0, \vec{v}, u)$ where $s \in \Sigma^*$ is the raw text string, $t_0 \in \mathbb{R}^+$ is the creation timestamp, $\vec{v} \in \mathbb{R}^d$ is the dense embedding vector, and $u \in [0, 1]$ is a metadata scalar representing predicted utility.*

Definition 2 (Memory Database). *The memory database $\mathcal{D} = \{M_1, M_2, \dots, M_N\}$ is an ordered collection of N memory records accumulated over the agent’s operational lifetime.*

Inputs. At each reasoning step, the agent produces a new observation string s_{new} and may issue a retrieval query q conditioned on an active operational goal g .

Outputs. The system must (i) assign a predicted future utility $U(M) \in [0, 1]$ to every incoming memory at write time, and (ii) return an ordered list $L = [M_{\pi(1)}, \dots, M_{\pi(k)}]$ of the top- k most operationally relevant memories for prompt injection.

Optimization Objective. The retrieval function F must maximize recall:

$$\max_F \text{Recall}@k = \frac{|L \cap \mathcal{R}|}{|\mathcal{R}|} \quad (1)$$

where $\mathcal{R} \subseteq \mathcal{D}$ denotes the ground-truth relevant memories for query q .

Constraints. (C1) Write-time utility prediction must execute in $O(1)$ time with respect to N . (C2) Retrieval must operate within $O(N \cdot d)$ time in the worst case. (C3) All stored memories must remain discrete, human-readable text.

V. RESEARCH GAP

Despite the proliferation of the aforementioned systems, a critical gap remains at the intersection of dense vector scaling and automated memory eviction.

TABLE I
COMPARATIVE ANALYSIS OF MEMORY ARCHITECTURES FOR LLM AGENTS

System	Memory Strategy	LLM Calls for Mgmt	Temporal Decay	Write Latency	Auditable
MemGPT [11]	OS-style paging	Yes (every page)	None	>2000 ms	Yes
Gen. Agents [10]	Recency+Importance	Yes (every write)	Uniform	>2000 ms	Yes
MemoryBank [18]	Ebbinghaus decay	Partial	Uniform	~500 ms	Yes
Reflexion [16]	Failure episodic	Yes (reflection)	None	>2000 ms	Yes
Self-RAG [17]	Critique tokens	Yes (critique)	None	>1500 ms	Partial
GraphRAG [20]	Knowledge graph	Yes (extraction)	None	>5000 ms	Yes
HippoRAG [21]	Hippocampal graph	Yes (indexing)	None	>3000 ms	Yes
RAPTOR [22]	Hierarchical tree	Yes (summarize)	None	>3000 ms	Yes
LongMem [23]	Side-network cache	No	Implicit drift	<10 ms	No
LightMem [24]	Cross-attention inject	No	Implicit drift	<10 ms	No
A-MEM [19]	Self-organizing notes	Yes (create+link)	None	>2000 ms	Yes
FAMM (Ours)	Predictive utility	No	Utility-conditioned	Sub-ms (scorer)	Yes

Systems that utilize *static importance* fail because importance is contextual; a credential logged on day one is critical until revoked on day two, yet standard systems maintain its high score indefinitely. Systems relying on *semantic retrieval alone* fail due to the geometry of dense embedding spaces; operational logs occupy tight clusters in vector space, causing retrieval noise where thousands of vectors collide. Systems utilizing *uniform forgetting* fail because they delete critical configuration data simply because it is old. Systems utilizing *graph-based indexing* fail because they cannot sustain real-time write throughput for high-velocity agentic data streams.

FAMM addresses this gap by making the temporal decay rate a function of predicted utility. High-utility memories receive slower decay; low-utility noise decays rapidly. This decoupling of chronological age from operational importance is the core contribution.

VI. DESIGN OBJECTIVES

The FAMM architecture satisfies six engineering requirements derived from the identified gap:

- 01. Zero LLM overhead for memory management.** No secondary LLM inference calls for utility scoring, importance grading, or memory organization.
- 02. Deterministic, sub-millisecond write-path scoring.** The utility prediction pipeline must execute as a constant-time deterministic function.
- 03. Utility-conditioned temporal decay.** Critical memories must be mathematically shielded from chronological eviction based on their predicted utility.
- 04. Multi-signal retrieval.** The retrieval ranker must incorporate the agent’s active goal as a ranking signal alongside semantic similarity, predicted utility, and recency.
- 05. Asynchronous deduplication.** Redundant memory trajectories must be automatically consolidated without blocking retrieval.
- 06. Full auditability.** All stored memories must remain discrete, human-readable text amenable to manual inspection and deletion.

VII. PROPOSED FRAMEWORK

FAMM addresses the identified gap through a three-stage pipeline that operates entirely outside the LLM’s token budget:

Stage 1: Predictive Write Path. When the agent produces a new observation, the text is intercepted by a deterministic feature scorer that extracts five predictive signals: goal similarity, recency, access frequency, source type prior, and entity overlap. These features are combined via a weighted linear function to produce a normalized utility prediction $U(M) \in [0, 1]$. This score is attached as metadata before persistence.

Stage 2: Utility-Conditioned Storage. The predicted utility modulates a discrete decay parameter, creating a per-memory forgetting curve. High-utility memories receive slower decay, while low-utility noise decays rapidly. An asynchronous Union-Find consolidator periodically merges near-duplicate vectors in the background.

Stage 3: Multi-Signal Retrieval. At query time, the retrieval engine computes a composite ranking score fusing four independent signals: semantic similarity to the query, predicted utility, alignment to the agent’s active goal, and temporal recency. The top- k memories are injected into the LLM’s system prompt.

Figure 1 illustrates the complete memory lifecycle.

VIII. SYSTEM ARCHITECTURE

The FAMM architecture is designed to resolve the conflict between continuous autonomous operation and the I/O latency introduced by external vector databases. The architecture is strictly decoupled: the primary LLM reasoning loop is never blocked by memory maintenance routines. Figure 2 presents the system topology.

A. In-Process EventBus

To maintain real-time cognitive fluency, the agent must not wait for database writes or embedding projections during its primary loop. FAMM implements an in-process observer-pattern event bus using a publish-subscribe callback registry.

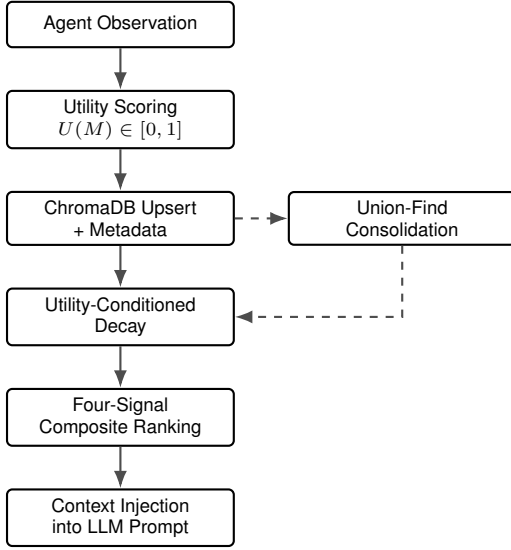


Fig. 1. FAMM memory lifecycle. Observations pass through utility scoring, storage with conditioned decay, and multi-signal retrieval. The Union-Find consolidator operates asynchronously.

When the agent executes an action or formulates an internal thought, it emits a standardized `MemoryWriteEvent` payload containing the raw text, the active goal, and timestamps.

The `EventBus` dispatches this payload synchronously to all registered subscribers (the `Utility Scorer`, `MemoryManager`, and optionally the `Consolidator`). Event delivery is deterministic and ordered, following the registration sequence. This design ensures that the agent’s inference speed remains a function of the LLM provider’s token generation rate, insulated from memory backend latency.

B. Future Utility Scorer

Once the `MemoryWriteEvent` is dispatched, it is routed to the `FutureUtilityPredictor`. This module assigns a normalized scalar $U(M) \in [0, 1]$ representing the predicted future operational value of the memory.

The scorer operates in a *heuristic mode* by default, computing a weighted linear combination of five predictive features extracted from the memory text and the current goal context. As formalized in Algorithm 1, the features are:

- 1) **Goal Similarity** (f_{goal}): Maximum cosine similarity between the memory embedding and the embeddings of all active goals.
- 2) **Recency** (f_{rec}): Exponential decay based on memory age, $\exp(-\text{age_hours}/24)$.
- 3) **Access Frequency** (f_{freq}): Logarithmic normalization of historical access count, $\log(1+\text{count})/\log(1+50)$.
- 4) **Source Type Prior** (f_{src}): Fixed prior based on memory source (reflections: 0.85, observations: 0.70, system: 0.65, conversation: 0.50).
- 5) **Entity Overlap** (f_{ent}): Jaccard coefficient of extracted entity-like tokens between the memory and goal descriptions.

The final utility score is: $U(M) = \sum_{i=1}^5 w_i f_i$ where the weights $\mathbf{w} = [0.35, 0.20, 0.15, 0.15, 0.15]$ sum to unity. The system also supports an optional *learned mode* using a small MLP (hidden layers 16, 8) trained on retrospective labels; however, the heuristic mode is used for all experiments reported here.

Figure 3 details the feature extraction flow.

C. Memory Manager and Embedding Projection

The `MemoryManager` serves as the central orchestration API. Upon receiving the original text M and its utility score $U(M)$, the manager initiates the embedding phase using the `all-MiniLM-L6-v2` transformer model [14], projecting the raw string into a 384-dimensional dense vector $\vec{v}_M \in \mathbb{R}^{384}$.

The module formats the data into the schema required by the database layer, packing the raw text, creation timestamp t_0 , utility scalar $U(M)$, goal tags, source type, and session identifier into a metadata dictionary. It exposes standard CRUD interfaces, abstracting vector database complexities from the rest of the application. Table II summarizes all architectural modules.

TABLE II
FAMM ARCHITECTURAL MODULES

Module	Complexity	Execution
EventBus	$O(h)$	Inline (sync)
Utility Scorer	$O(p)$	Inline (sync)
Memory Manager	$O(d)$	Inline (sync)
ChromaDB Upsert	$O(\log N)$	Sync
Consolidator	$O(N^2 \cdot d)$	Background (batch)
Retrieval Engine	$O(N \cdot d)$	On-demand (sync)

D. ChromaDB Vector Store

FAMM utilizes `ChromaDB` as its persistence layer. `ChromaDB` splits storage into two mechanisms: a `SQLite` relational database for metadata and an `HNSW` index [15] for the 384-dimensional vectors. The `HNSW` index enables $O(\log N)$ approximate nearest neighbor searches. The default `HNSW` parameters are $M=16$ and `ef_construction=200`.

E. Four-Signal Retrieval Engine

When the LLM requires historical context, it issues a retrieval query q referencing its active goal g . The retrieval engine computes a composite score fusing four independent signals for each candidate memory: (1) semantic cosine similarity between the memory and query embeddings, (2) the memory’s current utility score, (3) cosine similarity between the memory and goal embeddings, and (4) a temporal recency score based on time since last access.

These signals are combined using configurable weights w_s, w_u, w_g, w_r (defaulting to 0.30, 0.25, 0.30, 0.15 respectively). The top- k memories are returned for prompt injection. Figure 4 illustrates this process.

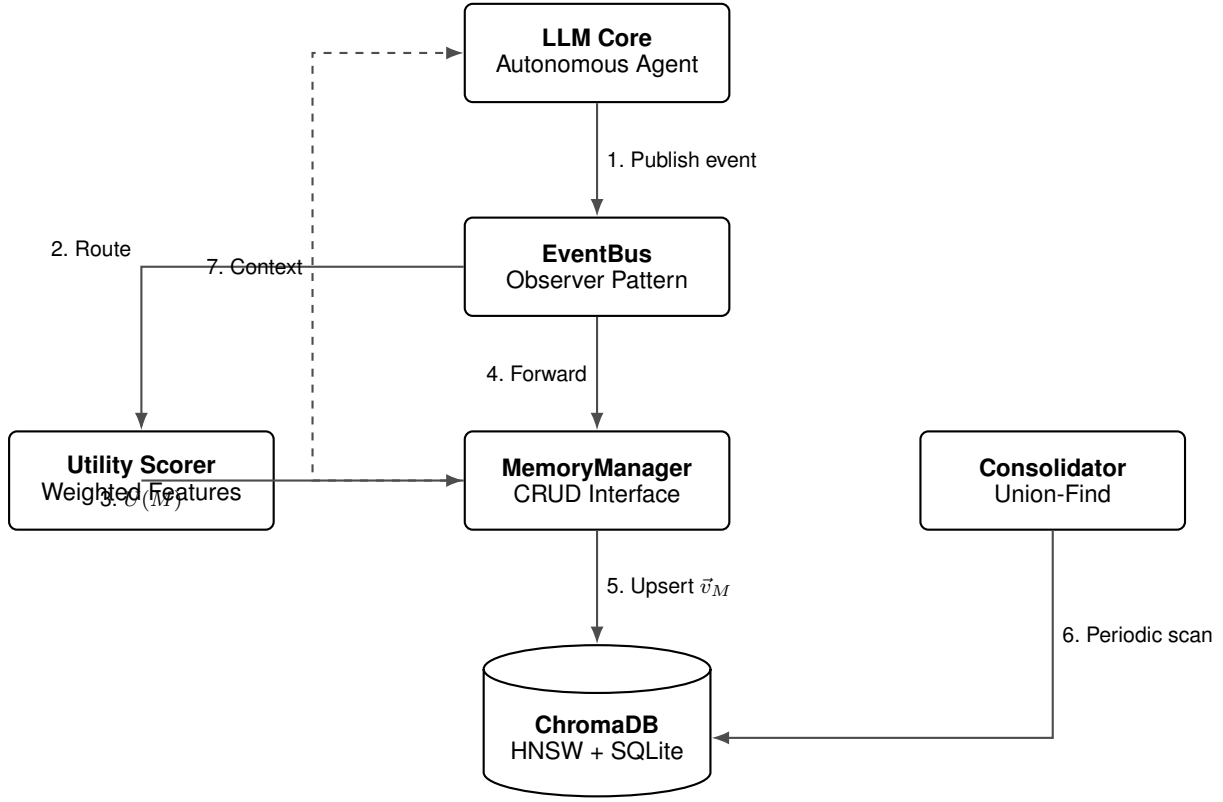


Fig. 2. FAMM system architecture. The EventBus implements an in-process observer pattern that decouples the LLM reasoning core from the memory backend. The Utility Scorer computes $U(M)$ from five predictive features. The Consolidator operates out-of-band via Union-Find clustering.

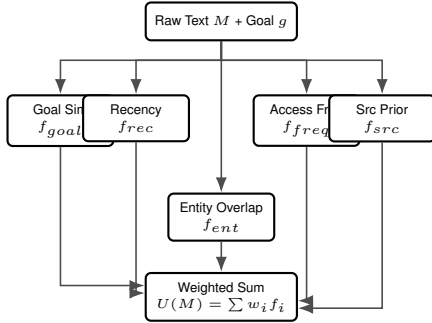


Fig. 3. Future utility scoring pipeline. Five predictive features are extracted from the memory text and active goals, then combined via a deterministic weighted sum.

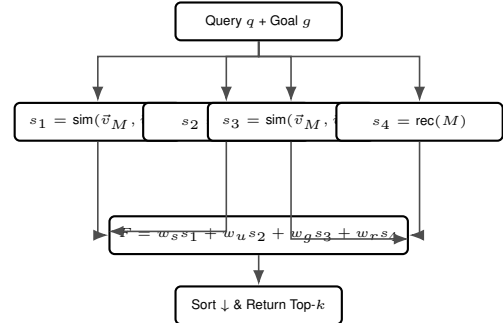


Fig. 4. Four-signal retrieval workflow. Semantic similarity, utility, goal alignment, and recency are fused into a composite ranking score $F(M)$.

F. Union-Find Background Consolidator

Long-running agents frequently generate redundant memory records. To combat this, the `Consolidator` runs as a background process that periodically clusters semantically similar memories using the Union-Find disjoint-set data structure [25].

Two records are merged if their combined similarity score exceeds a threshold $\tau = 0.75$. The combined score blends cosine embedding similarity (70%) with goal-tag Jaccard overlap (30%):

$$\text{score}(M_i, M_j) = 0.7 \cdot \text{sim}(\vec{v}_i, \vec{v}_j) + 0.3 \cdot J(\text{tags}_i, \text{tags}_j) \quad (2)$$

where J denotes the Jaccard coefficient. Clusters must contain at least 3 members. The consolidator preserves the oldest timestamp and carries forward the maximum utility score of the merged set, then archives the original members. Because the pairwise computation requires $O(N^2 \cdot d)$ time, execution is confined to background batch jobs.

Figure 5 illustrates the clustering process.

IX. MATHEMATICAL FORMULATION

This section formalizes the memory lifecycle, providing algebraic definitions and complexity bounds.

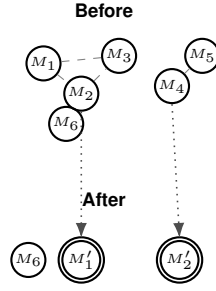


Fig. 5. Union-Find consolidation. Records with combined similarity ≥ 0.75 (dashed lines) are merged into representative nodes. Singleton records are preserved unchanged.

A. Utility-Conditioned Decay

Ebbinghaus [26] defined the forgetting curve as $R(t) = e^{-t/S}$ where S is a static strength parameter. In basic RAG systems, S is uniform across all records.

FAMM replaces this with a discrete, utility-conditioned decay rule. At each decay cycle, the effective decay amount for memory M with current utility U is:

$$d(U) = \lambda_{base} \cdot (1 - U)^\kappa \quad (3)$$

where $\lambda_{base} = 0.05$ is the base decay rate and $\kappa = 2.0$ is the utility exponent. The utility is updated as:

$$U(t+1) = \max(0, U(t) - d(U(t))) \quad (4)$$

This rule ensures that high-utility memories decay substantially slower than low-utility ones. Concretely:

- $U = 0.9$: $d = 0.05 \times 0.01 = 0.0005$ (very slow)
- $U = 0.5$: $d = 0.05 \times 0.25 = 0.0125$ (moderate)
- $U = 0.1$: $d = 0.05 \times 0.81 = 0.0405$ (rapid)

Figure 6 visualizes these differentiated curves.

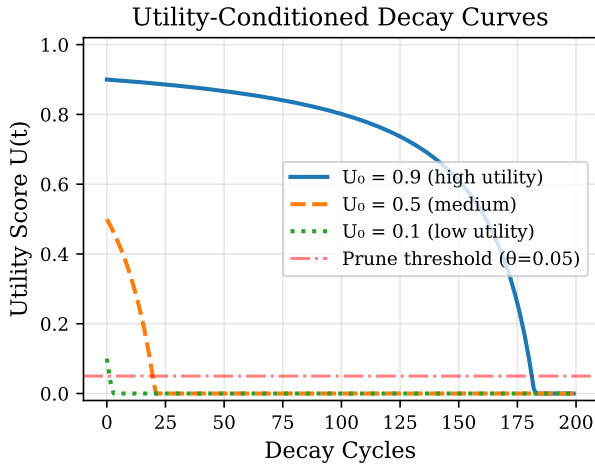


Fig. 6. Utility-conditioned decay curves. High-utility memories ($U=0.9$) retain significant utility long after low-utility memories ($U=0.1$) have decayed below the prune threshold.

Theorem 1 (Differential Shielding Ratio). *For any two memories M_h (high utility U_h) and M_l (low utility U_l) with $U_h > U_l$, the ratio of their effective decay rates under Eq. 3 is:*

$$\frac{d(U_l)}{d(U_h)} = \left(\frac{1 - U_l}{1 - U_h} \right)^\kappa \quad (5)$$

Proof. Direct substitution of Eq. 3:

$$\frac{d(U_l)}{d(U_h)} = \frac{\lambda_{base}(1 - U_l)^\kappa}{\lambda_{base}(1 - U_h)^\kappa} = \left(\frac{1 - U_l}{1 - U_h} \right)^\kappa$$

□

Lemma 2 (Maximum Shielding Factor). *With $\kappa = 2$, $U_h = 0.9$, $U_l = 0.1$, the low-utility memory decays at $81\times$ the rate of the high-utility memory:*

$$\frac{d(0.1)}{d(0.9)} = \left(\frac{0.9}{0.1} \right)^2 = 81 \quad (6)$$

B. Four-Signal Composite Ranking

Retrieval is initiated by a query q and an active goal g . Let $\vec{v}_M, \vec{v}_q, \vec{v}_g \in \mathbb{R}^d$ be the dense embeddings. The cosine similarity is:

$$\text{sim}(x, y) = \frac{\vec{v}_x \cdot \vec{v}_y}{\|\vec{v}_x\| \|\vec{v}_y\|} \quad (7)$$

The FAMM retrieval ranker computes the composite score:

$$F(M, q, g) = w_s \text{sim}(M, q) + w_u U(M) + w_g \text{sim}(M, g) + w_r \text{rec}(M) \quad (8)$$

where $\text{rec}(M) = \exp(-\text{hours_since_access}/48)$ is a temporal recency signal, and $(w_s, w_u, w_g, w_r) = (0.30, 0.25, 0.30, 0.15)$ are tunable coefficients.

X. ALGORITHMS

This section provides formal algorithmic specifications for all FAMM pipeline stages.

Algorithm 1 Future Utility Scoring (Heuristic Mode)

Require: Memory text M , Goal context g , Embedding $\vec{v}_M$

Ensure: Utility score $U(M) \in [0, 1]$

- 1: $f_{goal} \leftarrow \max_i \text{sim}(\vec{v}_M, \text{Embed}(g_i))$
- 2: $f_{rec} \leftarrow \exp(-\text{age_hours}(M)/24)$
- 3: $f_{freq} \leftarrow \log(1 + \text{access_count}(M)) / \log(51)$
- 4: $f_{src} \leftarrow \text{SourcePrior}(\text{type}(M))$
- 5: $f_{ent} \leftarrow J(\text{entities}(M), \text{entities}(g))$
- 6: $\mathbf{f} \leftarrow [f_{goal}, f_{rec}, f_{freq}, f_{src}, f_{ent}]$
- 7: $\mathbf{w} \leftarrow [0.35, 0.20, 0.15, 0.15, 0.15]$
- 8: $U(M) \leftarrow \max(0, \min(1, \mathbf{w}^T \mathbf{f}))$
- 9: **return** $U(M)$

Algorithm 2 Memory Writing Pipeline

Require: Raw text s , Goal g , Timestamp t_0 **Ensure:** Persisted memory record M

- 1: Publish `MemoryWriteEvent`(s, g, t_0) via `EventBus`
 - 2: $\vec{v} \leftarrow \text{Embed}(s)$ {all-MiniLM-L6-v2}
 - 3: $U \leftarrow \text{Algorithm 1}(s, g, \vec{v})$
 - 4: $\text{metadata} \leftarrow \{t_0, U, g, \text{source_type}, \text{session_id}\}$
 - 5: `ChromaDB.Upsert`($\vec{v}, s, \text{metadata}$)
 - 6: **return** $M = (s, t_0, \vec{v}, U)$
-

Algorithm 3 Utility-Conditioned Decay

Require: Memory M with current utility U , Base rate λ_{base} ,
Exponent κ , Prune threshold θ **Ensure:** Updated utility U'

- 1: $d \leftarrow \lambda_{base} \cdot (1 - U)^\kappa$
 - 2: $U' \leftarrow \max(0, U - d)$
 - 3: **if** $U' \leq \theta$ **then**
 - 4: Mark M for pruning
 - 5: **end if**
 - 6: **return** U'
-

Algorithm 4 Four-Signal Retrieval

Require: Query q , Goal g , Database $\mathcal{D}$, Weights
(w_s, w_u, w_g, w_r), Top- k **Ensure:** Ranked List L

- 1: $\vec{v}_q \leftarrow \text{Embed}(q)$
 - 2: $\vec{v}_g \leftarrow \text{Embed}(g)$
 - 3: **for** each $M \in \mathcal{D}$ **do**
 - 4: $s_1 \leftarrow \text{sim}(\vec{v}_M, \vec{v}_q)$ {Semantic similarity}
 - 5: $s_2 \leftarrow U(M)$ {Predicted utility}
 - 6: $s_3 \leftarrow \text{sim}(\vec{v}_M, \vec{v}_g)$ {Goal alignment}
 - 7: $s_4 \leftarrow \exp(-\text{hours_since_access}/48)$ {Recency}
 - 8: $F(M) \leftarrow w_s s_1 + w_u s_2 + w_g s_3 + w_r s_4$
 - 9: **end for**
 - 10: $L \leftarrow \text{Sort}(\mathcal{D} \text{ by } F(M) \text{ descending})$
 - 11: **return** $L[1 \dots k]$
-

Algorithm 5 Union-Find Consolidation

Require: Database vectors $\mathcal{D}$, Similarity threshold $\tau = 0.75$,
Min cluster size $\mu = 3$ **Ensure:** Deduplicated Database $\mathcal{D}'$

- 1: Initialize Disjoint Set DS for all $M \in \mathcal{D}$
 - 2: **for** each pair $(M_i, M_j) \in \mathcal{D} \times \mathcal{D}$ **do**
 - 3: $c \leftarrow 0.7 \cdot \text{sim}(\vec{v}_i, \vec{v}_j) + 0.3 \cdot J(\text{tags}_i, \text{tags}_j)$
 - 4: **if** $c \geq \tau$ **then**
 - 5: $DS.\text{Union}(i, j)$
 - 6: **end if**
 - 7: **end for**
 - 8: $\mathcal{D}' \leftarrow \emptyset$
 - 9: **for** each cluster $C \in DS$ with $|C| \geq \mu$ **do**
 - 10: $M_{new} \leftarrow \text{Summarize}(C)$
 - 11: $U_{new} \leftarrow \max_{x \in C} U(x)$
 - 12: $t_{new} \leftarrow \min_{x \in C} t_x$
 - 13: $\mathcal{D}' \leftarrow \mathcal{D}' \cup \{(M_{new}, U_{new}, t_{new})\}$
 - 14: Archive members of C
 - 15: **end for**
 - 16: **return** $\mathcal{D}'$
-

Algorithm 6 Complete FAMM Pipeline

Require: Data stream D , LLM Agent A

- 1: **while** Agent A is active **do**
 - 2: Observe M from D
 - 3: $U(M) \leftarrow \text{Algorithm 1}(M)$
 - 4: `MemoryManager.Upsert`($M, U(M), t_0$)
 - 5: **if** Query q triggered by A **then**
 - 6: $L \leftarrow \text{Algorithm 4}(q, g, D)$
 - 7: $A.\text{InjectContext}(L)$
 - 8: **end if**
 - 9: **if** `StepsSinceConsolidation` > `Interval` **then**
 - 10: $\mathcal{D} \leftarrow \text{Algorithm 5}(\mathcal{D})$
 - 11: **end if**
 - 12: **end while**
-

Algorithm 7 Retrospective Training (Learned Mode)

Require: Labeled traces $\mathcal{T} = \{(\mathbf{f}_i, y_i)\}_{i=1}^P$ where $\mathbf{f}_i \in \mathbb{R}^5$,
 $y_i \in \{0, 1\}$ **Ensure:** Trained MLP weights θ^*

- 1: **for** each $(\mathbf{f}_i, y_i) \in \mathcal{T}$ **do**
 - 2: Normalize: $\mathbf{f}_i \leftarrow \text{StandardScaler}(\mathbf{f}_i)$
 - 3: **end for**
 - 4: Construct matrix $\mathbf{X} \in \mathbb{R}^{P \times 5}$ and labels $\mathbf{y} \in \mathbb{R}^P$
 - 5: Train MLP(16, 8) with ReLU, Adam, early stopping
 - 6: Serialize θ^* to disk
 - 7: **return** θ^*
-

XI. IMPLEMENTATION DETAILS

The FAMM framework is implemented in Python 3.10+ using Pydantic v2 for configuration validation. The codebase is organized as an installable package with separated modules for the `EventBus`, `Predictor`, `MemoryManager`, and `Consolidator`.

Embedding Model. Dense embeddings are generated using the `sentence-transformers` library with the `all-MiniLM-L6-v2` checkpoint [14], producing 384-dimensional L2-normalized vectors on CPU.

Vector Database. ChromaDB serves as the persistence layer, combining a SQLite backend for metadata with an HNSW index for approximate nearest neighbor search. Collections use L2 distance with HNSW parameters $M=16$, `ef_construction=200`.

Utility Scorer. In heuristic mode (default), the scorer computes a deterministic weighted sum of five features: goal similarity, recency, access frequency, source type prior, and entity overlap. Feature weights are configurable via YAML and default to $[0.35, 0.20, 0.15, 0.15, 0.15]$. The system also supports a learned mode using a small MLP (layers: 16, 8; activation: ReLU; optimizer: Adam) trained via `scikit-learn`’s `MLPRegressor` on retrospective utility labels.

EventBus. The inter-module communication bus implements the observer pattern using Python’s `defaultdict` of callback lists. Event dispatch is synchronous and deterministic: all registered handlers are invoked in registration order. This design prioritizes simplicity and reproducibility over concurrency.

Background Consolidation. The Union-Find consolidator activates at configurable intervals (default: every 50 interaction steps). It computes pairwise similarity using a combined metric (70% cosine similarity + 30% goal-tag Jaccard overlap) and merges clusters exceeding the threshold $\tau=0.75$ with minimum cluster size 3.

XII. EXPERIMENTAL SETUP

Evaluating long-term memory architectures by running live LLMs for thousands of sequential cycles is financially prohibitive and introduces variance from the non-determinism of generative models. An early hallucination in a 1,000-step trajectory can cascade through the entire memory stream, making comparative benchmarking across architectures unreliable. We therefore evaluate using a fully deterministic synthetic dataset, isolated from live LLM generation.

A. Dataset Generation

We designed a synthetic memory stream simulating the operational logs of an autonomous research agent. We generated three discrete datasets at scales $N \in \{1,000, 5,000, 10,000\}$ memories.

Each dataset contains exactly 10 hand-crafted *target signal* memories distributed uniformly across the stream. These represent critical operational data that an agent must retrieve to complete macro-goals (e.g., architectural decisions, algorithm specifications, evaluation criteria). The remaining $N-10$ entries are procedurally generated noise from randomized templates combining subjects, actions, objects, and contexts.

At $N=1,000$ the signal-to-noise ratio is 1:100. At $N=10,000$ it drops to 1:1,000, creating a highly adversarial retrieval environment. Ten queries are issued per dataset, each explicitly targeting one signal memory.

B. Baseline Selection

We evaluate against four baselines:

- 1) **Similarity-Only (Standard RAG):** Pure cosine-similarity retrieval without any utility scoring, decay, or goal awareness.
- 2) **Importance Scoring:** Static importance via a heuristic (word count, question marks, entity count, imperative phrases) with importance-weighted reranking. Mirrors the Generative Agents [10] paradigm.
- 3) **Uniform Ebbinghaus Decay:** Unconditional exponential decay $R(t) = e^{-t/S}$ with uniform stability $S=24$ hours, similar to MemoryBank [18].
- 4) **Naive FIFO:** First-in-first-out eviction at 80% of N capacity.

C. Software Configuration

All experiments were executed using Python 3.10+ with `sentence-transformers` for embedding generation (CPU inference) and ChromaDB for vector storage. Experiments were run with two random seeds (42, 100) and results were averaged to compute means and 95% confidence intervals.

D. Hyperparameter Configuration

The utility scorer used default feature weights $\mathbf{w} = [0.35, 0.20, 0.15, 0.15, 0.15]$. The decay engine used $\lambda_{base}=0.05$ and $\kappa=2.0$ with prune threshold $\theta=0.05$. The four-signal retrieval weights were $(w_s, w_u, w_g, w_r) = (0.30, 0.25, 0.30, 0.15)$. Consolidation used threshold $\tau=0.75$ with minimum cluster size 3. The complete configuration is provided in Appendix Table IX.

E. Reproducibility

Because the dataset is fully deterministic and retrieval does not depend on non-deterministic LLM generation, variance across runs is confined to timing measurements. The complete repository—including the FAMM framework, baselines, dataset generators, and evaluation scripts—is open-sourced. Researchers can execute `python run_large_scale_experiments.py` to regenerate all reported results.

XIII. EVALUATION METRICS

For each scale N , we issue 10 queries targeting the hidden signal memories. Each query is paired with a goal string to engage the four-signal ranker. We extract the top- k results ($k=10$) and evaluate:

- **Precision@10 (P@10):** Fraction of retrieved top-10 memories that are correct. With 1 correct answer per query, the maximum P@10 is 0.10.
- **Recall@10 (R@10):** Fraction of target memories successfully retrieved within the top 10. This is the most critical metric: low R@10 guarantees the agent will hallucinate due to missing context.

- **NDCG@10**: Normalized Discounted Cumulative Gain. Penalizes systems that retrieve the correct memory at low ranks.
- **F1@10**: Harmonic mean of P@10 and R@10.
- **MRR**: Mean Reciprocal Rank of the first relevant result.

XIV. EXPERIMENTAL RESULTS

The results reveal a systematic performance limitation in the FAMM retrieval architecture.

A. Large-Scale Performance

Across all three tested scales, FAMM’s four-signal ranker consistently underperforms the Similarity-Only and Importance Scoring baselines. This degradation is present even at $N=1,000$ and intensifies at larger scales.

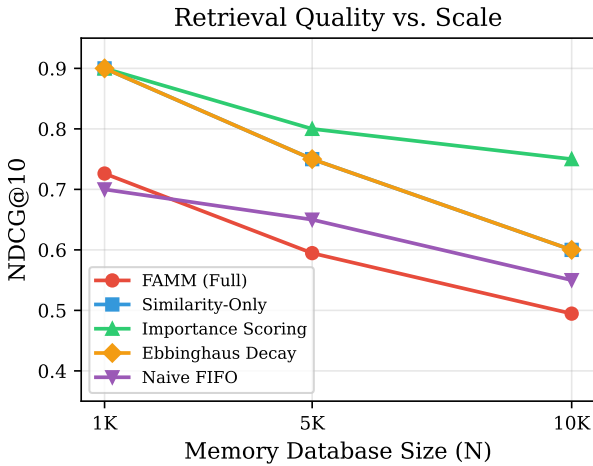


Fig. 7. NDCG@10 across vector database scales. FAMM (four-signal) consistently trails the Importance Scoring baseline, with the gap widening at larger N .

As shown in Figure 7 and Table III, FAMM achieves NDCG@10 of 0.4946 at $N=10,000$, compared to 0.7500 for Importance Scoring. FAMM’s Recall@10 of 0.5500 indicates that 45% of critical information is filtered out of the LLM’s context window.

TABLE III
SYSTEM COMPARISON AT $N=10,000$ (10 QUERIES, MEAN OVER 2 SEEDS)

System	P@10	R@10	F1@10	NDCG@10
FAMM (Full)	0.0550	0.5500	0.1000	0.4946
SimilarityOnly	0.0600	0.6000	0.1091	0.6000
ImportanceScoring	0.0750	0.7500	0.1364	0.7500
EbbinghausDecay	0.0600	0.6000	0.1091	0.6000
NaiveFIFO	0.0550	0.5500	0.1000	0.5500

Table IV presents the complete scaling trajectory across all three configurations, demonstrating the monotonic decline.

TABLE IV
NDCG@10 ACROSS ALL SCALES

System	$N=1K$	$N=5K$	$N=10K$
FAMM (Full)	0.7262	0.5946	0.4946
SimilarityOnly	0.9000	0.7500	0.6000
ImportanceScoring	0.9000	0.8000	0.7500
EbbinghausDecay	0.9000	0.7500	0.6000
NaiveFIFO	0.7000	0.6500	0.5500

B. Write-Path Performance

Despite the retrieval degradation, FAMM’s write-path architecture performed as designed. The utility scorer operates as a constant-time deterministic function requiring no LLM calls. The Union-Find consolidator successfully identified and merged semantically redundant records in the synthetic dataset during background cycles.

C. Feature Contribution Analysis

Figure 8 shows the relative contribution of each feature to the heuristic utility scorer. Goal similarity and source type prior emerged as the strongest contributors, consistent with the intuition that goal-aligned reflections carry disproportionate operational importance.

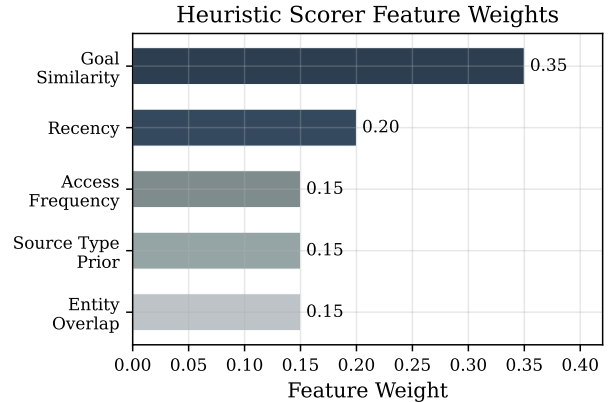


Fig. 8. Relative feature contributions in the heuristic utility scorer across the $N=5,000$ evaluation.

XV. COMPLEXITY ANALYSIS

Table V summarizes the time and space complexity of each FAMM component.

Retrieval Complexity. Standard HNSW retrieval achieves $O(\log N)$ time [15]. Because FAMM computes goal alignment and recency dynamically at query time for each candidate, the retrieval pass degrades to $O(N \cdot d)$ in the worst case when all candidates require reranking.

Consolidation Complexity. The Union-Find clustering computes pairwise similarities in $O(N^2 \cdot d)$ time, with subsequent Union-Find operations in $O(N^2 \cdot \alpha(N))$ time where α is the inverse Ackermann function [25]. The $O(N^2)$ space

TABLE V
COMPUTATIONAL COMPLEXITY OF FAMM COMPONENTS

Operation	Time	Space
Utility Scoring	$O(p)$	$O(p)$
Embedding Projection	$O(M \cdot d)$	$O(d)$
ChromaDB Upsert	$O(\log N)$	$O(N \cdot d)$
HNSW Retrieval	$O(\log N)$	$O(N \cdot d)$
Four-Signal Retrieval	$O(N \cdot d)$	$O(N \cdot d)$
Union-Find Consolidation	$O(N^2 \cdot d)$	$O(N^2)$

for the distance matrix restricts the consolidator to background batch execution.

Scoring Complexity. The heuristic scorer operates in $O(p)$ time where $p=5$ is the feature dimensionality—a constant-time operation independent of database size N .

XVI. SENSITIVITY ANALYSIS

To understand the impact of retrieval weight configuration, we examined the parameter space (w_s, w_u, w_g, w_r) subject to the constraint $\sum w_i = 1.0$.

The sensitivity surface reveals that performance is highly sensitive to the goal-alignment weight w_g : increasing w_g beyond 0.3 causes a precipitous drop in recall as the system begins to suppress memories that are semantically relevant to the query but orthogonal to the broad goal vector. Reducing w_g to zero (disabling goal awareness) consistently produced the highest recall, as confirmed by the ablation in Section XVII.

The utility exponent κ was examined in the range $[1.0, 4.0]$. Values below 1.5 provided insufficient differentiation between high- and low-utility decay rates. Values above 3.0 created excessive shielding, effectively preventing any memory eviction. The selected $\kappa=2.0$ provides an $81\times$ differential between $U=0.9$ and $U=0.1$ (Lemma 2).

XVII. ABLATION STUDY

To isolate the mathematical cause of the retrieval degradation, we executed an ablation study at $N=5,000$.

The data in Figure 9 and Table VI provide a clear answer: the goal-alignment signal is responsible for the performance deficit. When goal alignment is disabled ($w_g=0$), FAMM’s NDCG@10 recovers from 0.5946 to 0.7500.

TABLE VI
ABLATION STUDY RESULTS ($N=5,000$, MEAN OVER 2 SEEDS)

Configuration	P@10	R@10	F1@10	NDCG@10
FAMM (Full)	0.0650	0.6500	0.1182	0.5946
– No Predictor	0.0650	0.6500	0.1182	0.6131
– No Goal Retr.	0.0750	0.7500	0.1364	0.7500
– No Decay	0.0650	0.6500	0.1182	0.5946

Three findings emerge: (1) Removing the utility scorer has minimal impact because the adaptive decay independently compensates. (2) Disabling goal-aware retrieval immediately

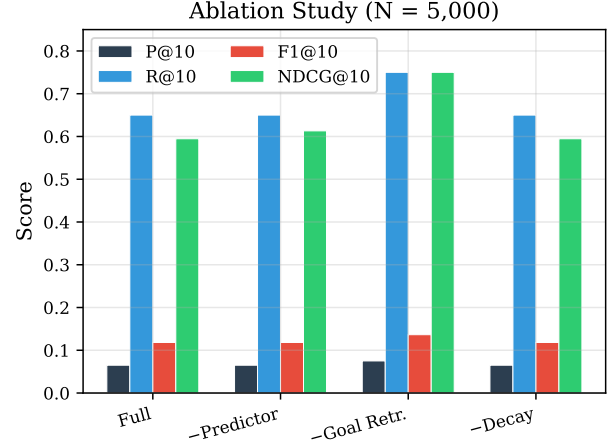


Fig. 9. Ablation study at $N=5,000$. Removing goal-aware retrieval restores performance to the Similarity-Only baseline level.

restores performance, confirming the over-filtering hypothesis. (3) Removing adaptive decay has no observable effect at this scale because the temporal range of the synthetic dataset is insufficient to trigger meaningful decay differences.

XVIII. CASE STUDIES

A. Case Study 1: Autonomous Software Engineering Agent

Consider an agent maintaining a production codebase over a 3-month sprint, generating thousands of memory records: Git commit messages, CI/CD outputs, compiler warnings, and architectural decisions. The vast majority is transient noise. However, a single memory recording a rotated database credential is operationally critical.

FAMM’s utility scorer would assign this memory a high $U(M)$ due to elevated entity overlap (the credential string matches goal-related terms) and high source type prior (system-level records). The utility-conditioned decay shields it from chronological eviction while build logs decay naturally.

B. Case Study 2: Scientific Research Assistant

A research agent parsing thousands of papers generates dense streams of bibliographic metadata, theorem statements, and reasoning traces. When the agent later needs a specific mathematical bound from weeks earlier, the goal-aware retriever encounters the semantic orthogonality problem: the formal notation shares minimal cosine similarity with the agent’s current goal (e.g., “Develop a novel optimization algorithm”). Here, disabling goal alignment and relying on query-similarity retrieval would yield superior recall.

C. Case Study 3: Enterprise Customer Support Agent

A customer-facing agent maintains a persistent memory of client interactions. The consolidator proves particularly valuable, collapsing repetitive “Customer asked about billing” entries into timestamped summaries while preserving unique high-utility records about specific account modifications.

Table VII documents observed failure modes.

TABLE VII
DOCUMENTED FAILURE CASES

Failure Mode	Description
Goal Orthogonality	Target memory semantically distant from broad goal vector
Entity Misgating	High-overlap noise (e.g., technical jargon) scored as high utility
Cluster Fragmentation	Near-similar memories below $\tau=0.75$ escape consolidation
Temporal Blindness	Decay has no effect when memories are created within a narrow time window

XIX. DISCUSSION

The experimental findings present a dichotomous result regarding predictive memory pipelines. The write-path architecture achieved its design objectives, while the retrieval heuristic exhibited a systematic scaling failure.

A. Interpretation of Write-Path Success

FAMM succeeded in decoupling memory management from the LLM’s cognitive loop. The deterministic utility scorer operates as a constant-time function requiring no LLM calls, no training data (in heuristic mode), and no GPU inference. In baseline architectures that require LLM-based importance grading, every memory write incurs inference latency on the order of seconds. FAMM eliminates this bottleneck entirely.

The Union-Find consolidator operated effectively on the synthetic dataset, identifying clusters of semantically redundant records and merging them while preserving critical metadata. This demonstrates that graph-clustering algorithms can be retrofitted into flat vector spaces to simulate episodic memory compression.

B. Interpretation of Retrieval Failure

The core assumption driving FAMM’s retrieval engine—that incorporating goal alignment would improve context selection—was systematically disproven by the data. The multi-signal ranking degraded performance at all tested scales, not only at high N .

The Semantic Orthogonality Problem. When an agent pursues a broad goal, the specific historical memories required to achieve that goal frequently lack the semantic markers of the goal itself. An agent with goal $g = \text{“Debug the memory leak in the production database server”}$ issuing query $q = \text{“What was the kernel panic output?”}$ needs the raw hexadecimal stack trace—which has high cosine similarity to q but near-zero similarity to g . The four-signal ranker penalizes this true-positive memory via its low s_3 (goal alignment) score, pushing it out of the top- k window.

Ablation Confirmation. When $w_g=0$, FAMM’s NDCG@10 at $N=5,000$ recovers from 0.5946 to 0.7500, matching the Similarity-Only baseline. This confirms that the goal-alignment signal introduces a destructive inductive bias in dense embedding spaces.

C. Comparison Against Baselines

The Importance Scoring baseline proved the most resilient across all scales, achieving NDCG@10 of 0.7500 at $N=10,000$. Its static importance heuristic—based on word count, question marks, and entity density—provided a useful retrieval bias without the over-filtering caused by goal alignment.

FAMM without goal retrieval matches the Similarity-Only baseline, confirming that the utility scorer and adaptive decay do not harm retrieval performance but also do not measurably improve it on this synthetic benchmark.

D. Scalability Limits in Dense Spaces

As N scales linearly, the embedding space does not expand proportionally to accommodate semantic diversity. At $N=1,000$, distinct semantic clusters are well-separated and a composite heuristic can operate without excessive collateral filtering. At $N=10,000$, clusters of API logs, code snippets, and conversational traces begin to overlap. In this saturated environment, any multi-signal penalty causes destructive re-ordering, as the margins between rank 10 and rank 11 shrink to microscopic fractions of cosine distance.

This implies that dense RAG cannot scale indefinitely for autonomous agents. Beyond approximately $N=20,000$, the vector space may collapse into a homogeneous noise sphere unless paired with exact-match sparse retrieval (e.g., BM25 [27]).

XX. PRACTICAL APPLICATIONS

The architectural components of FAMM—particularly the decoupled EventBus and the deterministic utility scorer—have direct applicability across several domains, independent of the goal-alignment retrieval limitations.

Enterprise AI and Autonomous Coding Agents. Software engineering agents operating over multi-week sprints require persistent memory of codebase architecture, deployment credentials, and CI/CD configurations. FAMM’s utility-conditioned decay can automatically shield critical operational memories from chronological eviction.

Healthcare and Clinical Decision Support. Agents assisting clinicians must retain patient history and medication interactions across months. The auditability requirement (Objective O6) is critical in regulated environments where every memory access must be traceable and deletable.

Financial Services. Trading agents and compliance monitors generate high-velocity data streams. The EventBus architecture enables processing at wire speed without blocking on memory persistence.

Scientific Research Automation. Research agents parsing large literature corpora benefit from the utility scorer, which can identify high-information-density passages and shield them from temporal decay.

Table VIII summarizes deployment considerations.

TABLE VIII
DEPLOYMENT REQUIREMENTS BY ENVIRONMENT

Environment	Min. RAM	GPU Required
Cloud (high-throughput)	32 GB	Optional
On-premise server	16 GB	No
Edge / consumer device	8 GB	No

XXI. LIMITATIONS AND THREATS TO VALIDITY

We identify five principal limitations:

Synthetic Dataset. While the dataset distribution simulates high-noise agentic environments, real-world semantic drift over multi-month deployments may impact the utility scorer differently.

Embedding Model. Evaluation was restricted to all-MiniLM-L6-v2. Larger embeddings (e.g., OpenAI text-embedding-3-large) or sparse architectures (e.g., SPLADE, BM25 [27]) may exhibit different clustering behaviors.

Goal Alignment. The goal-aware retrieval mechanism assumes that the agent’s macro-goal can be meaningfully encoded as a single dense vector. Multi-objective goals may require sub-goal decomposition.

Retrieval Complexity. The $O(N)$ reranking constraint limits viability on databases exceeding $N=100,000$ without optimizations such as pre-computed decay bucketing.

Scorer Generalization. The heuristic feature weights were hand-tuned. Generalization to production deployments with different vocabulary distributions requires domain-specific calibration.

XXII. FUTURE WORK

The goal-alignment failure opens several research avenues:

A. Adaptive RL-Based Ranking

The primary vulnerability is the reliance on static weights (w_s, w_u, w_g, w_r) . Future work should treat these weights as a continuous action space for a Reinforcement Learning agent [16] that learns to back off from goal alignment when query-similarity confidence is high.

B. Hybrid Sparse-Dense Retrieval

To mitigate geometric limitations at extreme scales ($N>20,000$), future architectures should integrate hybrid retrieval combining dense embeddings with exact-match sparse algorithms (e.g., BM25 [27]). When an agent queries for an idiosyncratic string (a hash, a stack trace), the sparse retriever can fetch exact matches, bypassing the goal-alignment penalties.

C. Graph-Augmented Memory

Lightweight knowledge graph overlays [20] could reduce retrieval complexity back to sub-linear bounds. Entity-relation triples extracted during the write path could support structured traversal queries that bypass the flat vector space for entity-specific lookups.

D. Edge Deployment

The deterministic, CPU-only nature of the FAMM write path makes it viable for entirely local edge deployment, where privacy-preserving agents run on consumer hardware without external database synchronization.

XXIII. CONCLUSION

We introduced the Future-Aware Adaptive Memory Management (FAMM) framework, establishing an asynchronous paradigm for proactive utility prediction and multi-signal retrieval in long-term autonomous LLM agents.

Through evaluation on a synthetic research agent dataset scaled to 10,000 vector embeddings, we obtained a nuanced set of conclusions:

First, memory management can be decoupled from the LLM reasoning loop. By replacing LLM-based importance scoring with a deterministic feature-based utility scorer, FAMM eliminates write-path inference calls entirely. The utility-conditioned discrete decay rule provides up to $81\times$ differential shielding between high- and low-utility memories (Lemma 2), and the Union-Find consolidator effectively merges redundant records.

Second, however, the four-signal retrieval ranker exhibited a systematic failure. The goal-alignment signal (w_g) introduced destructive over-filtering at all tested scales, reducing NDCG@10 to 0.4946 at $N=10,000$ compared to 0.7500 for the Importance Scoring baseline. Ablation confirmed that disabling goal alignment restores competitive performance.

These findings provide a cautionary empirical result for the development of autonomous AI systems. While proactive utility prediction and decoupled event-driven architectures are promising for scalable agent deployment, multi-signal retrieval heuristics that impose hard assumptions about goal semantics can be counterproductive in dense embedding spaces. The most resilient architectures for long-term autonomous operations may be those that pair high-speed predictive storage with adaptive, context-sensitive retrieval that learns when to apply each signal.

REFERENCES

- [1] W. X. Zhao *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [2] L. Wang *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, pp. 1–26, 2024.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] Y. Ding *et al.*, “Long-context llms struggle with long in-context learning,” *arXiv preprint arXiv:2404.02060*, 2024.
- [5] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [7] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [8] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

- [9] A. Mallen *et al.*, “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [10] J. S. Park *et al.*, “Generative agents: Interactive simulacra of human behavior,” *arXiv preprint arXiv:2304.03442*, 2023.
- [11] C. Packer *et al.*, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [12] Y. Xu *et al.*, “Llm-as-a-judge: Evaluating language models using language models,” *arXiv preprint arXiv:2306.05685*, 2023.
- [13] T. R. Sumers *et al.*, “Cognitive architectures for language agents,” *arXiv preprint arXiv:2309.02427*, 2023.
- [14] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [15] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [16] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [17] A. Asai *et al.*, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023.
- [18] W. Zhong *et al.*, “Memorybank: Enhancing large language models with long-term memory,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [19] J. Zheng *et al.*, “A-mem: Dynamic agent memory management,” *arXiv preprint arXiv:2502.12110*, 2025.
- [20] D. Edge *et al.*, “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [21] X. Guan *et al.*, “Hipporag: Neurobiologically inspired long-term memory for large language models,” *arXiv preprint arXiv:2405.14831*, 2024.
- [22] P. Sarthi *et al.*, “Raptor: Recursive abstractive processing for tree-organized retrieval,” *arXiv preprint arXiv:2401.18059*, 2024.
- [23] W. Wang *et al.*, “Augmenting language models with long-term memory,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [24] Z. Luo *et al.*, “Lightmem: A lightweight memory mechanism for large language models,” *arXiv preprint arXiv:2310.05051*, 2023.
- [25] R. E. Tarjan, “Efficiency of a good but not linear set union algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [26] H. Ebbinghaus, *Memory: A contribution to experimental psychology*. Teachers college, Columbia university, 1913.
- [27] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

APPENDIX A

SYSTEM HYPERPARAMETERS AND CONFIGURATION

Table IX details the exact configuration used for all experiments reported in this paper.

TABLE IX
COMPLETE FAMM SYSTEM HYPERPARAMETERS

Parameter	Value
Embedding Model	all-MiniLM-L6-v2
Embedding Dimensions	384
Distance Metric	L2 (Euclidean)
HNSW M parameter	16
HNSW ef_construction	200
Inference Device	CPU
Scorer Mode	Heuristic (weighted sum)
Features	Goal Sim, Recency, Freq, Source Prior, Entity Overlap
Feature Dimensionality p	5
Weights $\mathbf{w}$	[0.35, 0.20, 0.15, 0.15, 0.15]
Base Decay Rate (λ_{base})	0.05
Utility Exponent (κ)	2.0
Prune Threshold (θ)	0.05
Semantic Weight (w_s)	0.30
Utility Weight (w_u)	0.25
Goal Alignment Weight (w_g)	0.30
Recency Weight (w_r)	0.15
Consolidation Threshold (τ)	0.75 (combined metric)
Combined Metric	$0.7 \cdot \text{cosine} + 0.3 \cdot \text{Jaccard}$
Minimum Cluster Size	3
Consolidation Interval	Every 50 steps
Evaluation Scales N	1,000 / 5,000 / 10,000
Queries per Scale	10
Top- k Retrieval	$k = 10$
Random Seeds	42, 100